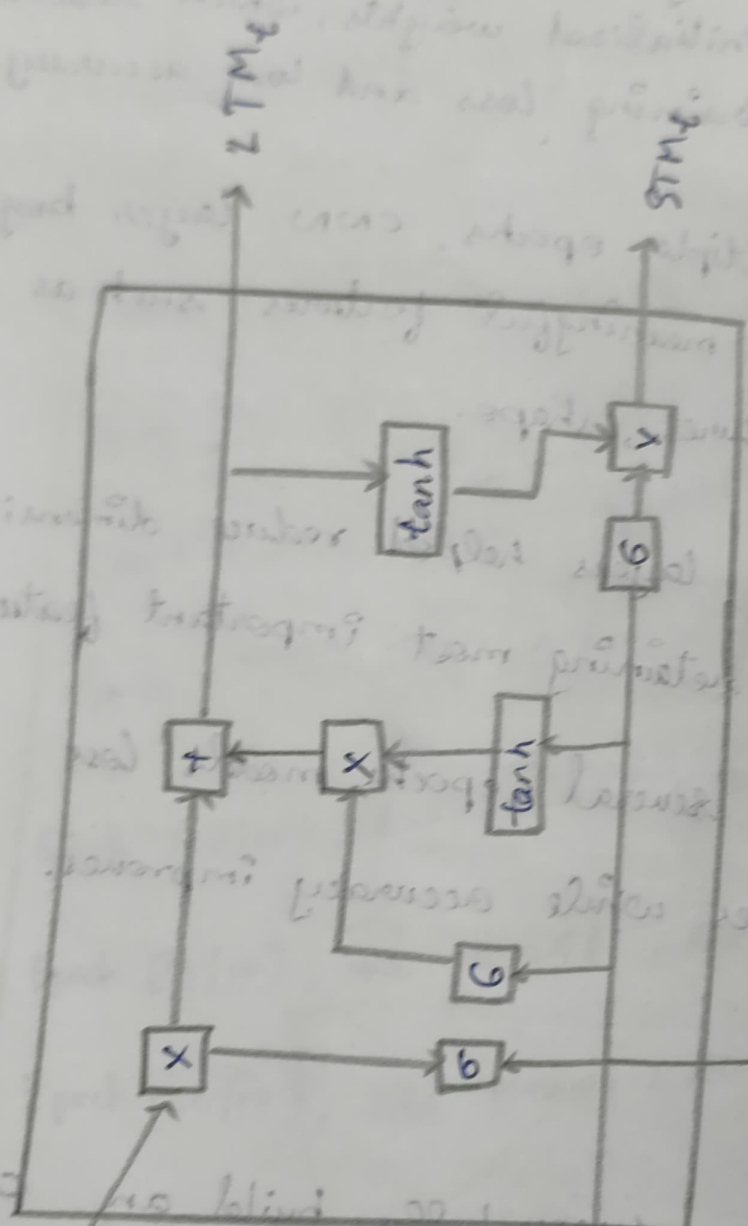


LSTM

Architecture

LSTM

LSTM



AIM:

To do experiment using Long-term-Short-term-memory network.

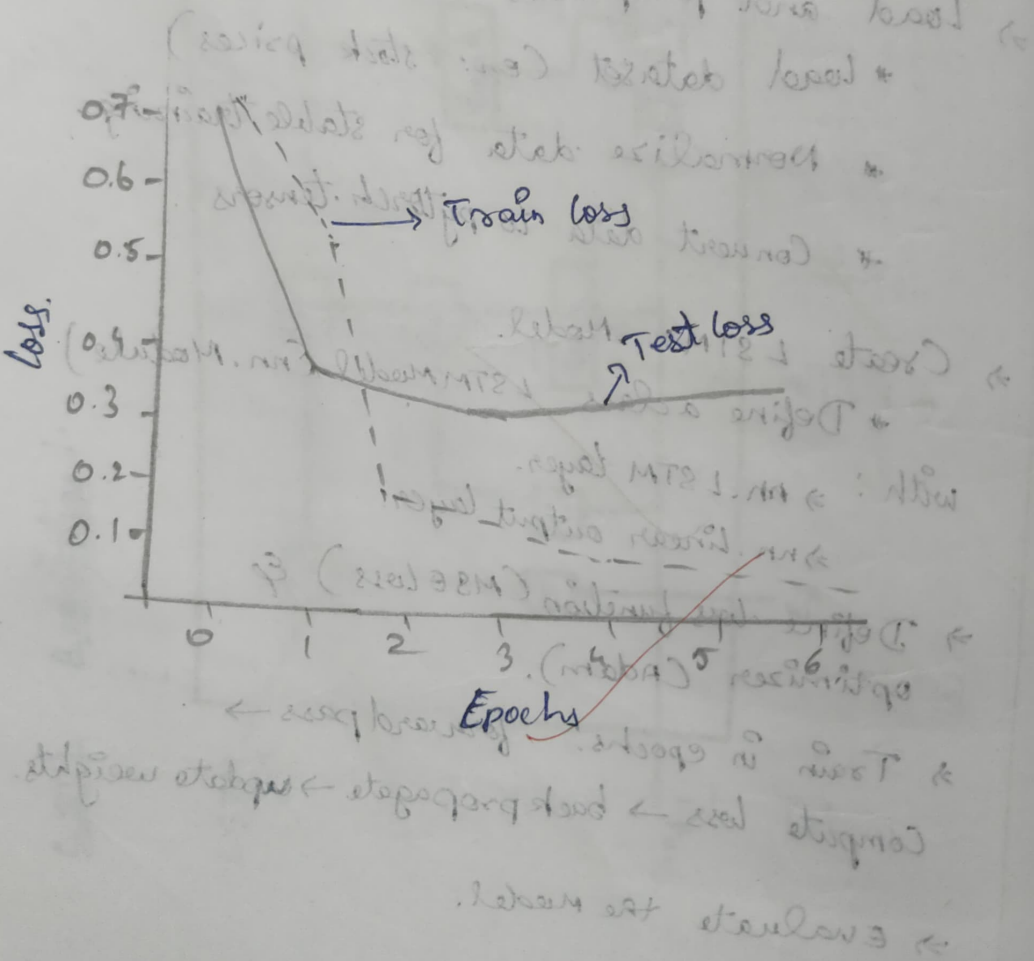
ALGORITHM:

- Import the required libraries such as numpy, pandas, tensorflow, matplotlib, torch, torch.nn, torch.optim.
- Load and preprocess the Data.
 - * load dataset (ex: stock prices)
 - * Normalize data for stable training.
 - * Convert data to pytorch tensors.
- Create LSTM Model.
 - * Define a class LSTMModel (nn.Module) with:
 - nn.LSTM layer.
 - nn.Linear output layer.
- Define loss function (MSE loss) & optimizer (Adam).
- Train in epochs.
 - forward pass →
 - Compute loss → back propagate → update weights.
- Evaluate the Model.

Epochs:

Epoch 1/6	Train loss: 0.6120	Test loss: 0.6478
Epoch 2/6	Train loss: 0.5677	Test loss: 0.3960
Epoch 3/6	Train loss: 0.2993	Test loss: 0.3175
Epoch 4/6	Train loss: 0.1969	Test loss: 0.3175
Epoch 5/6	Train loss: 0.1360	Test loss: 0.2907
Epoch 6/6	Train loss: 0.0780	Test loss: 0.3000

Train / Test loss Graph.



PSEUDOCODE:

IMPORT torch, numpy, matplotlib.

LOAD and NORMALIZE dataset.

CREATE sequences (x, y)

SPLIT into train and test.

CONVERT to torch Tensors.

DEFINE LSTM Model with:

LSTM layer + linear layer

FORWARD: pass $x \rightarrow \text{LSTM} \rightarrow \text{last hidden} \rightarrow \text{linear}$.

INITIALIZE model, MSE Loss, Adam optimizer.

For each epoch:

model.train()

y-pred = model(x-train)

loss = MSE (y-pred, y-train)

optimizer.zero_grad()

loss.backward()

optimizer.step()

model.eval()

WITH no_grad:

predictions = model(x-test)

CALCULATE RMSE

PLOT training loss.

PLOT actual vs predicted values.

OBJECTIVE:

- * To learn how to build and train an LSTM model in Pytorch.
- * To prepare sequential data and feed it into an LSTM network.
- * To visualize training loss and prediction results.

OBSERVATION:

- * The training loss decreases steadily, showing that the model is learning.
- * The predicted curve closely follows the actual sine wave, demonstrating that the LSTM captured temporal dependencies.
- * Compared to simple feedforward network, LSTM performs better for sequential data.

RESULT:

Therefore the experiment using LSTM is completed successfully.